

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра Програмної інженерії

Звіт
з лабораторної роботи №1
з дисципліни: «Високорівневі мови програмування та фреймворки»
з теми: «Django»

Виконав:
ст. гр. ПЗПІ-23-2
Ситник Є. С.
Варіант: №12

Перевірів:
ст. викл. кафедри ПІ
Саманцов О. О.

1 DJANGO

1.1 Мета роботи

Метою даної роботи є ознайомлення з основами розробки за допомогою веб-фреймворку Python Django. Розгляд архітектурного патерну MVC, проектування моделей даних, написання контролерів для обробки запитів та використання мови шаблонів для відображення інформації користувачу.

1.2 Індивідуальне завдання

- Рівень 1: Створіть Django-модель «Категорія» з полями «Назва» та «Опис». Забезпечте можливість призначення статті до конкретної категорії.
- Рівень 2: Створіть Django-модель «Коментар» з полями «Автор», «Текст» та «Дата коментування». Забезпечте можливість додавання коментарів до статей.
- Рівень 3: Додайте пагінацію для списку статей, щоб користувач міг легко переглядати багато статей на одній сторінці.
- Рівень 4: Реалізуйте можливість пошуку статей за ключовими словами. Додайте функціонал пошуку до веб-інтерфейсу.

1.3 Хід роботи

1.3.1 Рівень 1.

```
@final
class Category(m.Model):
    name = m.CharField(max_length=255, verbose_name="Назва")
    description = m.TextField(blank=True, verbose_name="Опис")

    @override
    def __str__(self):
        return self.name
```

1.3.2 Рівень 2.

```
@final
class Comment(m.Model):
    post = m.ForeignKey(
        Post,
        on_delete=m.CASCADE,
```

```

        related_name="comments",
        verbose_name="Стаття",
    )
    author = m.ForeignKey(User, on_delete=m.CASCADE,
        verbose_name="Автор")
    text = m.TextField(verbose_name="Текст коментаря")
    created_at = m.DateTimeField(auto_now_add=True,
        verbose_name="Дата коментування")

    @override
    def __str__(self):
        return f"Коментар від {self.author} до {self.post.title}"

```

1.3.3 Рівні 3 та 4.

```

def article_list(request: HttpRequest) -> HttpResponse:
    search_query = request.GET.get("q", "")
    posts = Post.objects.all().order_by("-published")

    if search_query:
        posts = posts.filter(
            Q(title__icontains=search_query) |
            Q(content__icontains=search_query)
        )

    paginator = Paginator(posts, 5)
    page_number = request.GET.get("page")
    page_obj = paginator.get_page(page_number)

    context = {
        "page_obj": page_obj,
        "search_query": search_query,
    }

    return render(request, "blog/article_list.html", context)

```

1.4 Висновки

Під час виконання даної роботи було розглянуто архітектурний патерну MVC, спроектовано моделі даних, написано контролер для обробки запитів та використано мову шаблонів для відображення інформації користувачу.